

```
%% MAIN RADAR DIAGNOSTIC SIMULATION WITH ALGORITHM TELEMETRY CONSOLE
```

```
clear; clc; close all;
```

```
%% 1. ENVIRONMENT & UAV PATH SETUP
```

```
fprintf('Initializing Physical Environment...\n');
```

```
fsamp = 10;
```

```
t_end = 5;
```

```
time_vec = (0:1/fsamp:t_end)';
```

```
num_steps = length(time_vec);
```

```
% Target Tree Setup [X, Y, Z]
```

```
tree_position = [25, 3, 0];
```

```
% UAV Flight Path (Constant Altitude)
```

```
waypoints = [0 0 5; 35 0 5];
```

```
toa = [0; t_end];
```

```
uav_positions = interp1(toa, waypoints, time_vec, 'linear');
```

```
uav_velocities = zeros(num_steps, 3);
```

```
uav_velocities(1:end-1, :) = diff(uav_positions) * fsamp;
```

```
uav_velocities(end, :) = uav_velocities(end-1, :);
```

```
% Open an expansive dashboard window with room for text telemetry
```

```
fig = figure('Name', 'MIMO Radar Physical & Algorithmic Diagnostic Dashboard',  
'Position', [30, 30, 1350, 800]);
```

```
%% 2. SIMULATION PROCESSING LOOP
```

```
for k = 1:num_steps
```

```
    uav_pos = uav_positions(k, :);
```

```
    uav_vel = uav_velocities(k, :);
```

```
    yaw_rad = atan2(uav_vel(2), uav_vel(1));
```

```
    %% PHYSICS ENGAGEMENT: RUN RADAR PROCESSING CHAIN
```

```
    [clean_points, rd_map, range_axis, vel_axis, spatial_spectrum, scan_angles] = ...  
        radar_signal_processor(uav_pos, uav_vel, yaw_rad, tree_position);
```

```
%% 3. GENERATE DYNAMIC 4-PANEL PHYSICS VISUALIZATION + CONSOLE
```

```
clf;
```

```
%% PANEL 1: RADAR WAVE TRANSMISSION PROFILE
```

```
subplot(2, 3, 1);
```

```
[X_grid, Y_grid] = meshgrid(0:1:40, -15:1:15);
```

```
beam_angle = atan2d(Y_grid, X_grid);
```

```
distance_grid = sqrt(X_grid.^2 + Y_grid.^2);
```

```
beam_intensity = exp(-(beam_angle.^2)/(2*20^2)) ./ (distance_grid + 1).^0.5;
```

```
imagesc(0:40, -15:15, beam_intensity); hold on;
```

```
rel_tree = tree_position - uav_pos;
```

```
plot(rel_tree(1), rel_tree(2), 'ro', 'MarkerSize', 12, 'LineWidth', 3);
```

```

title('1. Radar Wave Transmission Field');
xlabel('Distance Ahead (m)'); ylabel('Lateral Spread (m)');
axis xy;

%% PANEL 2: RECEIVED ECHO ENERGY (RANGE-DOPPLER HEATMAP)
subplot(2, 3, 2);
imagesc(vel_axis, range_axis, 20*log10(abs(rd_map)));
axis xy; colormap(gca, 'parula');
title('2. Received Echo Profile (Range-Doppler)');
xlabel('Relative Velocity (m/s)'); ylabel('Range (m)');
ylim([0 40]);

%% PANEL 3: DIGITAL BEAMFORMING SPEC
subplot(2, 3, 4);
plot(scan_angles, 20*log10(spatial_spectrum/max(spatial_spectrum)), 'LineWidth', 2, 'Color', [0.85 0.33 0.1]);
grid on;
title('3. Digital Beamforming Spectrum');
xlabel('Azimuth Angle (Deg)'); ylabel('Response (dB)');
ylim([-25 0]); xlim([-60 60]);

%% PANEL 4: 3D SPATIAL PERCEPTION MAP
subplot(2, 3, 5);
[X_g, Y_g] = meshgrid(-5:5:45, -15:5:15);
mesh(X_g, Y_g, zeros(size(X_g)), 'FaceColor', 'none', 'EdgeColor', [0.8 0.8 0.8]); hold on;
plot3([tree_position(1) tree_position(1)], [tree_position(2) tree_position(2)], [0 3], 'Color', [0.4 0.2 0], 'LineWidth', 4);
scatter3(tree_position(1), tree_position(2), 3, 300, 'g', 'filled');
scatter3(uav_pos(1), uav_pos(2), uav_pos(3), 150, 'b', 'filled', '^');
plot3(uav_positions(:,1), uav_positions(:,2), uav_positions(:,3), 'b:');

if ~isempty(clean_points)
    R_rot = [cos(yaw_rad), -sin(yaw_rad), 0; sin(yaw_rad), cos(yaw_rad), 0; 0, 0, 1];
    glob_det = (R_rot * clean_points)' + uav_pos;
    scatter3(glob_det(:,1), glob_det(:,2), glob_det(:,3), 100, 'magenta', 'LineWidth', 2);
end
title('4. Fused 3D Spatial Perception Map');
xlabel('X (m)'); ylabel('Y (m)'); zlabel('Z (m)');
view(45, 20); grid on; axis equal;
xlim([-5 40]); ylim([-15 15]); zlim([0 8]);

%% 🖥️ DYNAMIC ALGORITHMIC TELEMETRY CONSOLE PANEL
% This places a clean, live updating text panel on the right side of the window
subplot(2, 3, [3, 6]);
axis off;

```

```

% Compute relative physics metrics for live updates
current_dist = norm(tree_position - uav_pos);
current_vel = norm(uav_vel);

% Construct detailed algorithmic walkthrough text array
telemetry_text = {
    '=====
    '          UAV MIMO RADAR SYSTEM TELEMETRY          '
    '=====
    sprintf(' Flight Frame: %d / %d', k, num_steps)
    sprintf(' UAV Velocity: %.2f m/s | Altitude: %.1f m', current_vel, uav_pos)
(3)    sprintf(' Dist to Target Tree: %.2f meters', current_dist)
    '-----'
    ' [STEP 3-4] WAVEFORM TRANSMISSION & ECHO:'
    '   - Radar Type: 77 GHz Frequency Modulated Continuous Wave (FMCW).'
    '   - Signal: Chirp sawtooth pulse sweeping over 300 MHz bandwidth.'
    '   - Reflection: Radar wave illuminates the tree foliage canopy.'
    '     The rough surface causes Diffuse Backscattering, scattering'
    '     electromagnetic energy echo arrays backward through space.'
    ''
    ' [STEP 5-7] ANALOG MIXING & DIGITAL DSP PIPELINE:'
    '   - Operation: Echo mixed with original Tx to get intermediate beat.'
    '   - Fast-Time FFT: Extracts exact time delay -> Range Profile.'
    '   - Slow-Time FFT: Analyzes phase variation between consecutive '
    '     chirps -> Isolates Doppler Frequency (Relative Speed).'
    ''
    ' [STEP 8-11] MIMO ARRAYS & VIRTUAL BEAMFORMING:'
    '   - Setup: 4 Tx Transmitters * 4 Rx Receivers architecture.'
    '   - Coarray Synthesis: Forms a 16-element Virtual Linear Array.'
    '   - Algorithm: Conventional Spatial Fourier Beamforming / MUSIC Principle.'
    '     Computes steering vectors a(theta) for scanning angles -60 to +60.'
    '     Locates incoming Angle-of-Arrival (AoA) phase shift maximum.'
    ''
    ' [STEP 13] RADAR PREPROCESSING (CLUTTER FILTERING):'
    '   - Method: Threshold Amplitude Masking (Emulating CA-CFAR).'
    '   - Action: Strips random thermal noise spikes and multipath anomalies.'
    '     Converts raw beat frequencies into crisp (X, Y, Z, V) Point Cloud.'
    '=====
};

% Render the text to the display screen box
text(0.02, 0.95, telemetry_text, 'FontName', 'Monospaced', 'FontSize', 10, ...
    'VerticalAlignment', 'top', 'BackgroundColor', [0.96 0.96 0.96], ...
    'EdgeColor', [0.8 0.8 0.8]);

drawnow;
pause(0.1);
end

```

```

function [clean_point_cloud, rd_map, range_axis, vel_axis, spatial_spectrum, ↵
scan_angles] = ...
    radar_signal_processor(uav_pos, uav_vel, yaw_rad, tree_position)

% Advanced Radar Hardware Setup
fc = 77e9; c = 3e8; lambda = c/fc;
num_virtual = 16;
bw = 300e6; tm = 50e-6; slope = bw/tm;
num_samples = 256; num_chirps = 32;
fs = 2*bw;
scan_angles = -60:0.5:60;
num_angles = length(scan_angles);

adc_cube = zeros(num_samples, num_chirps, num_virtual);
t_fast = linspace(0, tm, num_samples)';

% Transform to local UAV frame
R_yaw = [cos(yaw_rad), sin(yaw_rad), 0; -sin(yaw_rad), cos(yaw_rad), 0; 0, 0, 1];
rel_pos_local = (R_yaw * (tree_position - uav_pos)')';
r = norm(rel_pos_local);
az = atan2d(rel_pos_local(2), rel_pos_local(1));

fprintf(' [STEP 3-4] RF WAVEFORM TRANSMISSION & BACKSCATTERING:\n');
fprintf('    -> Emitting 77 GHz LFM CW sawtooth waveforms.\n');

if rel_pos_local(1) > 0 && r < 45 && abs(az) < 60
    fprintf('    -> Target tree hit! Rough foliage surface causes Diffuse ↵
Backscattering.\n');

% Formulate Phase shifting vector across array elements
sv = exp(1i * 2 * pi * (0:num_virtual-1)' * 0.5 * sind(az));
v_rel = dot(uav_vel, (tree_position - uav_pos))/r;

for chirp = 1:num_chirps
    t_slow = (chirp - 1) * tm;
    delay = 2 * (r - v_rel * t_slow) / c;
    beat_freq = slope * delay;
    doppler_freq = 2 * v_rel * fc / c;

% Physics Base Wave equation
signal = exp(1i * 2 * pi * (beat_freq * t_fast + doppler_freq * t_slow));
    for ch = 1:num_virtual
        adc_cube(:, chirp, ch) = adc_cube(:, chirp, ch) + signal * sv(ch);
    end
end
end

% Controlled noise injection to prevent vertical map saturation stripes

```

```

    adc_cube = adc_cube + 0.01 * (randn(size(adc_cube)) + 1i * randn(size(
(adc_cube)));
    fprintf(' [STEP 5] ADC DIGITIZATION: Signal quantized.\n');

%% STEP 6 & 7: 2D WINDOWED FFT CONVOLUTION
fprintf(' [STEP 6-7] DIGITAL SIGNAL PROCESSING CHAINS:\n');

% Force strong Blackman-Harris windows to clear side-lobe leakage completely
win_fast = blackman(num_samples);
win_slow = blackman(num_chirps);
windowed_cube = zeros(size(adc_cube));
for ch = 1:num_virtual
    windowed_cube(:, :, ch) = (adc_cube(:, :, ch) .* win_fast) .* win_slow';
end

range_fft = fft(windowed_cube, num_samples, 1);
rd_map_full = fftshift(fft(range_fft, num_chirps, 2), 2);
rd_map = sum(abs(rd_map_full), 3);

range_axis = ((0:num_samples-1) * (fs/num_samples) * c) / (2 * slope);
doppler_axis = linspace(-1/(2*tm), 1/(2*tm), num_chirps);
vel_axis = (doppler_axis * c) / (2 * fc);

%% STEP 8 - 11: VIRTUAL ARRAY PROCESSING
fprintf(' [STEP 8-11] MIMO ARRAYS & VIRTUAL BEAMFORMING:\n');

[~, max_idx] = max(rd_map(:));
[r_bin, d_bin] = ind2sub(size(rd_map), max_idx);
snapshots = squeeze(rd_map_full(r_bin, d_bin, :));

spatial_spectrum = zeros(num_angles, 1);
for a = 1:num_angles
    sv_look = exp(1i * 2 * pi * (0:num_virtual-1)' * 0.5 * sind(scan_angles(a)));
    spatial_spectrum(a) = abs(sv_look' * snapshots)^2;
end

[~, max_ang_idx] = max(spatial_spectrum);
est_az = scan_angles(max_ang_idx);
est_rng = range_axis(r_bin);

fprintf('    -> Beamformer lock: Target resolved at angle %.1f degrees.\n',
est_az);

%% STEP 13: PREPROCESSING & POINT CLOUD FILTERING
fprintf(' [STEP 13] THRESHOLD PREPROCESSING FILTERS:\n');
clean_point_cloud = [];

% Dynamically adjusted detection floor cutoff
if rd_map(r_bin, d_bin) > (mean(rd_map(:)) * 2) && r < 45

```

```

        fprintf('    -> Target resolved: Range = %.2f m | Azimuth = %.1f deg.\n', est_rng, est_az);
    clean_point_cloud = [est_rng * cosd(est_az), est_rng * sind(est_az), 0];
else
    fprintf('    -> No clear targets detected out of the noise floor yet.\n');
end
end
function unified_features = ai_feature_fusion(img, point_cloud)
    %% STEP 14: CAMERA PROCESSING & ENCODING
    img_resized = imresize(img, [64, 64]);

    % Native Convolution Matrix Filter (Replaces heavy Deep Learning Toolbox requirement)
    edge_kernel = [-1 -1 -1; -1 8 -1; -1 -1 -1];
    visual_features = double(imfilter(img_resized, edge_kernel));

    %% STEP 15: RADAR FEATURE EXTRACTION
    if ~isempty(point_cloud)
        radar_features = mean(point_cloud, 1);
    else
        radar_features = [0, 0, 0];
    end

    %% STEP 16 & 17: CROSS-MODAL FUSION & DENSE MATRIX GENERATION
    unified_features = visual_features(:, :, 1) + radar_features(1);
    fprintf('    [AI] Run: Combined vision edge maps with spatial radar matrices.\n');
end
function next_step = navigation_planner(uav_pos, clean_point_cloud)
    %% STEP 18 & 19: OCCUPANCY GRID MAPPING
    local_grid = zeros(10, 10);

    if ~isempty(clean_point_cloud)
        grid_x = ceil(clean_point_cloud(:, 1));
        grid_y = ceil(clean_point_cloud(:, 2)) + 5;

        for i = 1:length(grid_x)
            if grid_x(i) > 0 && grid_x(i) <= 10 && grid_y(i) > 0 && grid_y(i) <= 10
                local_grid(grid_x(i), grid_y(i)) = 1;
            end
        end
        fprintf('    [Nav] Occupancy Map calculated successfully.\n');
    end

    %% STEP 20: PATH PLANNING
    avoidance_vector = [1, 0, 0];
    if ~isempty(clean_point_cloud)
        closest_obstacle = clean_point_cloud(1, :);
        if closest_obstacle(2) > 0
            avoidance_vector = [1, -0.5, 0]; % Veer Right
        end
    end
end

```

```
        else
            avoidance_vector = [1, 0.5, 0]; % Veer Left
        end
    end
    next_step = uav_pos + avoidance_vector;
end
```